

Name and Surname: Mogomotsi More

Student ITS No: 402411435

Qualification: Bachelor of Science in Information Technology

Year of Study: 3

Semester: 1

Assignment due date: 28 May 2026

Date submitted: 28 May 2026

QUESTION	EXAMINER MARKS	MODERATOR MARKS	REMARKS

ASSIGNMENT INSTRUCTIONS

Please tick each box to confirm completion.

Use Times New Roman font, size 12, with 1.5-line spacing throughout the document. Apply Harvard Referencing Style for all citations and references.

For essay-style assignments, please include the following sections:

Table of Contents

Introduction

Main Body (with relevant subheadings)

Conclusion

References

Submit the assignment in PDF format on Moodle.

Use the specified cover page provided.

Include a signed declaration of originality.

DECLARATION OF ORIGINALITY:

I hereby declare that this assignment is my own work and has not been copied from any other source except where due acknowledgment is made. I affirm that all sources used have been properly cited and that this submission complies with the institution's policies on academic integrity and plagiarism.

Student Signature: _____



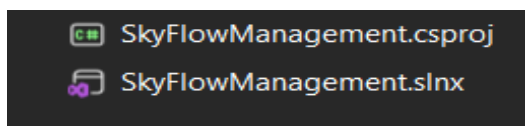
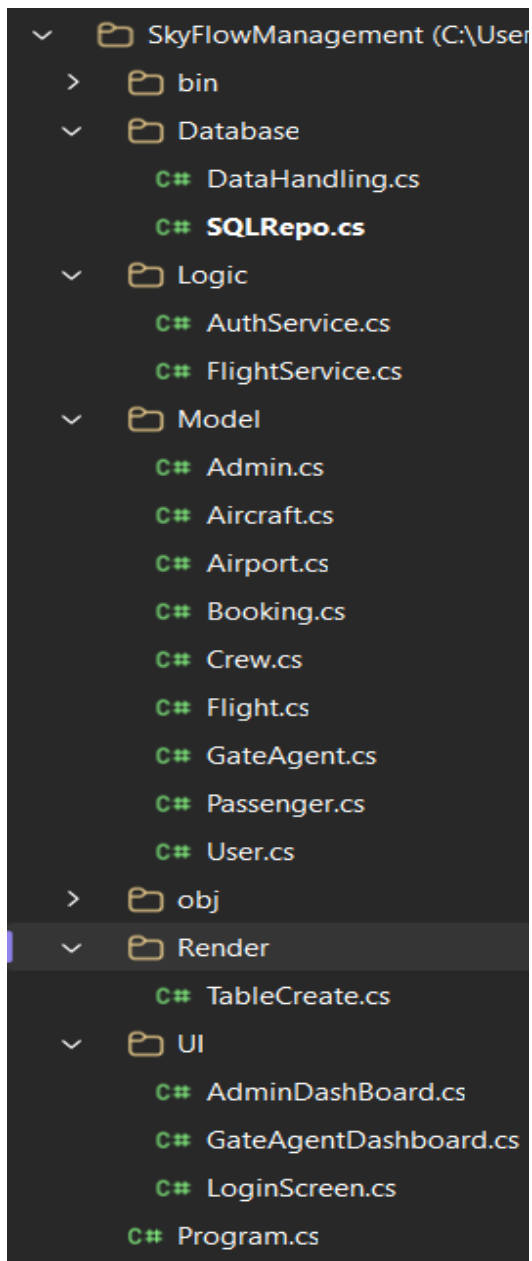
Date: 28 May 2026

Contents

Folder/File Architecture:.....	4
User.cs.....	5
Admin.cs	6
GateAgent.cs	7
.....	7
Flight.cs.....	8
.....	8
.....	8
Passenger.cs	9
.....	9
Booking.cs	10
.....	10
.....	10
Aircraft.cs.....	11
.....	11
Airport.cs	12
.....	12
Crew.cs.....	13
.....	13
DataHandling.cs.....	14
SQLRepo.cs	15
FlightService.cs.....	21
AuthServices.cs.....	26
TableCreate.cs.....	27
LoginScreen.cs	29
AdminDashBoard.cs	30
GateAgentDashBaord.cs	32
Program.cs	34
CreateTable.sql	35

seedData.sql	38
--------------------	----

Folder/File Architecture:



User.cs

```
1  using System;
2  using System.Collections.Generic;
3  using System.Text;
4
5  namespace SkyFlow.Models
6  {
7      public abstract class User
8      {
9          // Properties:
10         public int UserId { get; private set; }
11         public string Username { get; private set; }
12         public string Role { get; private set; }
13         public string Email { get; private set; }
14         public string FirstName { get; private set; }
15         public string LastName { get; private set; }
16         public DateTime CreatedAt { get; private set; }
17         private string PasswordHash { get; set; }
18
19         // Constructor:
20         protected User(int userId, string username, string passwordHash, string role, string email, string firstName, string lastName, DateTime createdAt)
21         {
22             UserId = userId;
23             Username = username;
24             PasswordHash = passwordHash;
25             Role = role;
26             Email = email;
27             FirstName = firstName;
28             LastName = lastName;
29             CreatedAt = createdAt;
30         }
31         // Method:
32
33         public bool validatePassword(string Hash)
34         {
35             return PasswordHash == Hash;
36         }
37
38         // Template for displaying menu
39         public abstract void displayDashboard();
40     }
```

Admin.cs

```
1  using System;
2  using System.Collections.Generic;
3  using System.Text;
4
5  namespace SkyFlow.Models
6  {
7      public class Admin : User
8      {
9          // Base construtor
10         public Admin(int userId, string username, string passwordHash, string email, string firstName, string lastName, DateTime createdAt)
11             : base(userId, username, passwordHash, "Admin", email, firstName, lastName, createdAt)
12         { }
13
14
15         // Method:
16
17         // overriding the method template from the user class.
18         public override void displayDashboard()
19         {
20             Console.WriteLine("Welcome " + FirstName + " !");
21             Console.WriteLine(" Admin Dashboard");
22             Console.WriteLine(" 1. Manage Flights");
23             Console.WriteLine(" 2. View System Overview");
24             Console.WriteLine(" 3. Manage Staff");
25             Console.WriteLine(" 0. Logout");
26         }
27     }
28 }
29
```

GateAgent.cs

```
1  using System;
2  using System.Collections.Generic;
3  using System.Text;
4
5  namespace SkyFlow.Models
6  {
7      public class GateAgent : User
8      {
9          // Base constructor:
10         public GateAgent(int userId, string username, string passwordHash, string email, string firstName, string lastName, DateTime createdAt)
11             : base(userId, username, passwordHash, "GateAgent", email, firstName, lastName, createdAt)
12         { }
13
14         //Method:
15
16         // overriding the template for this method from the users class.
17         public override void displayDashboard()
18         {
19             Console.WriteLine("Welcome " + FirstName + " !");
20             Console.WriteLine(" Gate Agent Dashboard");
21             Console.WriteLine(" 1. View Flight Manifest");
22             Console.WriteLine(" 2. Passenger Check-In");
23             Console.WriteLine(" 3. Boarding Gate");
24             Console.WriteLine(" 0. Logout");
25         }
26     }
27 }
28
```

Flight.cs

```
1  using System;
2  using System.Collections.Generic;
3  using System.Security;
4  using System.Security.Cryptography.X509Certificates;
5  using System.Text;
6
7  namespace SkyFlow.Models
8  {
9      public class Flight
10     {
11         // Property:
12         public int FlightId { get; private set; }
13         public string FlightNumber { get; private set; }
14         public string Origin { get; private set; }
15         public string Destination { get; private set; }
16         public DateTime DepartureTime { get; private set; }
17         public DateTime ArrivalTime { get; private set; }
18         public int Capacity { get; private set; }
19         public int CurrentOccupancy { get; private set; }
20         public string Status { get; private set; }
21         public int GateAgentId { get; private set; }
22
23         // Constructor:
24         public Flight(int flightId, string flightNumber, string origin, string destination, DateTime departureTime, DateTime arrivalTime, int capacity, int currentOccupancy, string status, int gateAgentId)
25         {
26             FlightId = flightId;
27             FlightNumber = flightNumber;
28             Origin = origin;
29             Destination = destination;
30             DepartureTime = departureTime;
31             ArrivalTime = arrivalTime;
32             Capacity = capacity;
33             CurrentOccupancy = currentOccupancy;
34             Status = status;
35             GateAgentId = gateAgentId;
36         }
37
38         // Methods:
39
40         public bool IsFull()
41         {
42             if (CurrentOccupancy >= Capacity)
43             {
44                 return true;
```

```
45         }
46         else
47         {
48             return false;
49         }
50     }
51
52     public void updateOccupancy(int count)
53     {
54         CurrentOccupancy = count;
55     }
56
57     public void departFlight()
58     {
59         if (Status == "Departed")
60         {
61             Console.WriteLine(" Flight has already departed.");
62             return;
63         }
64         Status = "Departed";
65     }
66
67     public void updateStatus(string newStatus)
68     {
69         Status = newStatus;
70     }
71 }
72 }
```

Passenger.cs

```
1  using System;
2  using System.Collections.Generic;
3  using System.Text;
4
5  namespace SkyFlow.Models
6  {
7      public class Passenger
8      {
9          public int PassengerId { get; private set; }
10         public int UserId { get; private set; }
11         public string PassportNumber { get; private set; }
12         public DateTime DateOfBirth { get; private set; }
13         public string Nationality { get; private set; }
14         public string ContactNumber { get; private set; }
15
16         public Passenger(int passengerId, int userId, string passportNumber, DateTime dateOfBirth, string nationality, string contactNumber)
17         {
18             PassengerId = passengerId;
19             UserId = userId;
20             PassportNumber = passportNumber;
21             DateOfBirth = dateOfBirth;
22             Nationality = nationality;
23             ContactNumber = contactNumber;
24         }
25     }
26 }
```

Booking.cs

```
1 using System;
2 using System.Collections.Generic;
3 using System.Reflection;
4 using System.Text;
5
6 namespace SkyFlow.Models
7 {
8     public class Booking
9     {
10         // Properties:
11
12         public int BookingId { get; private set; }
13         public int FlightId { get; private set; }
14         public int PassengerId { get; private set; }
15         public string SeatNumber { get; private set; }
16         public string BookingStatus { get; private set; }
17         public DateTime BookingDate { get; private set; }
18         public DateTime? CheckInTime { get; private set; }
19         public DateTime? BoardingTime { get; private set; }
20
21         // Constructor:
22
23         public Booking(int bookingId, int flightId, int passengerId, string seatNumber, string bookingStatus, DateTime bookingDate, DateTime? checkInTime, DateTime? boardingTime)
24         {
25             BookingId = bookingId;
26             FlightId = flightId;
27             PassengerId = passengerId;
28             SeatNumber = seatNumber;
29             BookingStatus = bookingStatus;
30             BookingDate = bookingDate;
31             CheckInTime = checkInTime;
32             BoardingTime = boardingTime;
33         }
34
35         //Method:
36
37         public void updateStatus(string newStatus)
38         {
39             BookingStatus = newStatus;
40         }
41     }
42 }
```

```
41
42         public void checkIn()
43         {
44             BookingStatus = "CheckedIn";
45             CheckInTime = DateTime.Now;
46         }
47
48         public void board()
49         {
50             BookingStatus = "Boarded";
51             BoardingTime = DateTime.Now;
52         }
53     }
54 }
```

Aircraft.cs

```
1  using System;
2  using System.Collections.Generic;
3  using System.Text;
4
5  namespace SkyFlow.Models
6  {
7      public class Aircraft {} namespace SkyFlow.Models
8      {
9          // Properties:
10
11          public int AircraftId { get; private set; }
12          public string AircraftType { get; private set; }
13          public string RegistrationNumber { get; private set; }
14          public string Manufacturer { get; private set; }
15          public int Capacity { get; private set; }
16          public string Status { get; private set; }
17
18          // Constructor:
19
20          public Aircraft(int aircraftId, string aircraftType, string registrationNumber, string manufacturer, int capacity, string status)
21          {
22              AircraftId = aircraftId;
23              AircraftType = aircraftType;
24              RegistrationNumber = registrationNumber;
25              Manufacturer = manufacturer;
26              Capacity = capacity;
27              Status = status;
28          }
29
30          //Methods:
31
32          public void updateStatus(string newStatus)
33          {
34              Status = newStatus;
35          }
36      }
37  }
```

Airport.cs

```
1  using System;
2  using System.Collections.Generic;
3  using System.Text;
4
5  namespace SkyFlow.Models
6  {
7      public class Airport
8      {
9          public int AirportId { get; private set; }
10         public string AirportCode { get; private set; }
11         public string AirportName { get; private set; }
12         public string City { get; private set; }
13         public string Country { get; private set; }
14
15         public Airport(int airportId, string airportCode, string airportName, string city, string country)
16         {
17             AirportId = airportId;
18             AirportCode = airportCode;
19             AirportName = airportName;
20             City = city;
21             Country = country;
22         }
23     }
24 }
```

Crew.cs

```
1  using System;
2  using System.Collections.Generic;
3  using System.Text;
4
5  namespace SkyFlow.Models
6  {
7      public class Crew
8      {
9          public int CrewId { get; private set; }
10         public int UserId { get; private set; }
11         public string CrewType { get; private set; }
12         public string LicenseNumber { get; private set; }
13         public int YearsOfExperience { get; private set; }
14         public string Status { get; private set; }
15
16         public Crew(int crewId, int userId, string crewType, string licenseNumber, int yearsOfExperience, string status)
17         {
18             CrewId = crewId;
19             UserId = userId;
20             CrewType = crewType;
21             LicenseNumber = licenseNumber;
22             YearsOfExperience = yearsOfExperience;
23             Status = status;
24         }
25
26         public void updateStatus(string newStatus)
27         {
28             Status = newStatus;
29         }
30     }
31 }
```

DataHandling.cs

```
1  using SkyFlow.Models;
2
3  namespace SkyFlow.Interfaces
4  {
5      // List of methods that will be used by other class specifically within the UI directory.
6      // The are current empty till used.
7
8      public interface IDataRepository
9      {
10         // --- User ---
11
12         // Finds a user in the database by their username else return a null value.
13         User? getUserByUsername(string username);
14
15         // Adds a new staff member to the database.
16         void addUser(string username, string passwordHash, string role, string email, string firstName, string lastName);
17
18
19         // --- Flights ---
20
21         // Gets every flight in the system as a list.
22         List<Flight> getAllFlights();
23
24         // Looks for specific flight depending on flight number, else return a null value
25         Flight? getFlightByNumber(string flightNumber);
26
27         // Adds a brand new flight to the database.
28         void addFlight(string flightNumber, string origin, string destination, DateTime departureTime, DateTime arrivalTime, int capacity, int gateAgentId);
29
30         // Changes a flight's status e.g. from "Scheduled" to "Boarding".
31         void updateFlightStatus(string flightNumber, string newStatus);
32
33
34         // --- Passengers & Bookings ---
35
36         // Gets all passengers on a specific flight.
37         // Added together cause you need both when boarding.
38         List<(Passenger Passenger, Booking Booking)> getPassengersByFlightNumber(string flightNumber);
39
40         // Search's for one specific passenger on a flight
41         (Passenger Passenger, Booking Booking)? getPassengerBySearch(string flightNumber, string search);
42
43         // Updates a passenger's booking status e.g. "CheckedIn" or "Boarded".
44         // implemented without data till an action is taken.
45         void updateBookingStatus(int bookingId, string newStatus, DateTime? checkInTime, DateTime? boardingTime);
46
47
48         // --- Aircraft ---
49
50         // Gets a list of all aircrafts in the database.
51         List<Aircraft> getAllAircraft();
52
53
54         // --- Airport ---
55
56         // Gets a list of all airports in the database.
57         List<Airport> getAllAirports();
58
59
60         // --- Crew ---
61
62         // Gets a list of all crew members.
63         List<Crew> getAllCrew();
64
65         // Adds a new crew member to the database.
66         void addCrew(int userId, string crewType, string licenseNumber, int yearsOfExperience);
67     }
68 }
```

SQLRepo.cs

```
1  using Microsoft.Data.SqlClient;
2  using SkyFlowManagement.Model;
3
4  namespace SkyFlowManagement.Database
5  {
6      // SQLRepo communicate with the database.
7      // It implements all the methods stored in the Datahadling class
8      // which is responsible for any database manipulation
9
10     public class SQLRepo : DataHandling
11     {
12         // The connection string that tells the system where to find the SQL Server database
13         private readonly string _connectionString;
14
15         public SQLRepo(string connectionString)
16         {
17             _connectionString = connectionString;
18         }
19         //Creates a new database connection.
20         private SqlConnection getConnection() => new SqlConnection(_connectionString);
21
22
23         // *****
24         //         --- User Methods ---
25         // *****
26
27         public User? getUserByUsername(string username)
28         {
29             // Open a connection to the database ad closes automaticallly after implementation.
30             using var conn = getConnection();
31             conn.Open();
32
33             // Write the SQL query
34             // @Username acts as a placeholder.
35             using var cmd = new SqlCommand("SELECT * FROM Users WHERE Username = @Username", conn);
36             cmd.Parameters.AddWithValue("@Username", username);
37
```

```

38 using var reader = cmd.ExecuteReader();
39
40 // If no user was found, return null.
41 if (!reader.Read()) return null;
42
43 // Gets role so we know which sub-class to implement.
44 string role = reader["Role"].ToString() ?? "";
45
46 // Gets informatio from the database and stores it in variables:
47 int userId = (int)reader["UserId"];
48 string uname = reader["Username"].ToString() ?? "";
49 string passwordHash = reader["PasswordHash"].ToString() ?? "";
50 string email = reader["Email"].ToString() ?? "";
51 string firstName = reader["FirstName"].ToString() ?? "";
52 string lastName = reader["LastName"].ToString() ?? "";
53 DateTime createdAt = (DateTime)reader["CreatedAt"];
54
55 // Calls sub-class depending on the role
56 return role == "Admin"
57     ? new Admin(userId, uname, passwordHash, email, firstName, lastName, createdAt)
58     : new GateAgent(userId, uname, passwordHash, email, firstName, lastName, createdAt);
59 }
60
61 public void addUser(string username, string passwordHash, string role, string email, string firstName, string lastName)
62 {
63     using var conn = GetConnection();
64     conn.Open();
65
66     // Added placeholders for cleaner use of data.
67     using var cmd = new SqlCommand(@"
68         INSERT INTO Users (Username, PasswordHash, Role, Email, FirstName, LastName, CreatedAt)
69         VALUES (@Username, @PasswordHash, @Role, @Email, @FirstName, @LastName, @CreatedAt)", conn);
70
71     cmd.Parameters.AddWithValue("@Username", username);
72     cmd.Parameters.AddWithValue("@PasswordHash", passwordHash);

```

```

73     cmd.Parameters.AddWithValue("@Role", role);
74     cmd.Parameters.AddWithValue("@Email", email);
75     cmd.Parameters.AddWithValue("@FirstName", firstName);
76     cmd.Parameters.AddWithValue("@LastName", lastName);
77     cmd.Parameters.AddWithValue("@CreatedAt", DateTime.Now);
78
79     // Allows us to perform basic database manipulation without returning a row of data.
80     // Just shows us which row was affected or add into the database as a integer.
81     cmd.ExecuteNonQuery();
82 }
83
84
85 // *****
86 // --- Flight Methods ---
87 // *****
88
89 public List<Flight> getAllFlights()
90 {
91     // Start with an empty list and fill it as we read rows.
92     var flights = new List<Flight>();
93
94     using var conn = getConnection();
95     conn.Open();
96
97     using var cmd = new SqlCommand("SELECT * FROM Flights", conn);
98     using var reader = cmd.ExecuteReader();
99
100     // Reads data from the database till the last row of the database.
101     while (reader.Read())
102     {
103         flights.Add(mapFlight(reader));
104     }
105
106     return flights;
107 }
108
109 public Flight? getFlightByNumber(string flightNumber)

```

```

110 {
111     using var conn = getConnection();
112     conn.Open();
113
114     using var cmd = new SqlCommand("SELECT * FROM Flights WHERE FlightNumber = @FlightNumber", conn);
115     cmd.Parameters.AddWithValue("@FlightNumber", flightNumber);
116
117     using var reader = cmd.ExecuteReader();
118
119     if (!reader.Read()) return null;
120     return mapFlight(reader);
121 }
122
123 public void addFlight(string flightNumber, string origin, string destination, DateTime departureTime, DateTime arrivalTime, int capacity, int gateAgentId)
124 {
125     using var conn = getConnection();
126     conn.Open();
127
128     using var cmd = new SqlCommand(@"
129         INSERT INTO Flights (FlightNumber, Origin, Destination, DepartureTime, ArrivalTime, Capacity, CurrentOccupancy, Status, GateAgentId)
130         VALUES (@FlightNumber, @Origin, @Destination, @DepartureTime, @ArrivalTime, @Capacity, 0, 'Scheduled', @GateAgentId)", conn);
131
132     cmd.Parameters.AddWithValue("@FlightNumber", flightNumber);
133     cmd.Parameters.AddWithValue("@Origin", origin);
134     cmd.Parameters.AddWithValue("@Destination", destination);
135     cmd.Parameters.AddWithValue("@DepartureTime", departureTime);
136     cmd.Parameters.AddWithValue("@ArrivalTime", arrivalTime);
137     cmd.Parameters.AddWithValue("@Capacity", capacity);
138     cmd.Parameters.AddWithValue("@GateAgentId", gateAgentId);
139
140     cmd.ExecuteNonQuery();
141 }
142
143 public void updateFlightStatus(string flightNumber, string newStatus)
144 {
145     using var conn = getConnection();

```

```

144 {
145     using var conn = getConnection();
146     conn.Open();
147
148     using var cmd = new SqlCommand(@"
149         UPDATE Flights
150         SET Status = @Status
151         WHERE FlightNumber = @FlightNumber", conn);
152
153     cmd.Parameters.AddWithValue("@Status", newStatus);
154     cmd.Parameters.AddWithValue("@FlightNumber", flightNumber);
155
156     cmd.ExecuteNonQuery();
157 }
158
159
160 // *****
161 //      --- Passenger & Booking Methods ---
162 // *****
163
164 public List<Passenger Passenger, Booking Booking> getPassengersByFlightNumber(string flightNumber)
165 {
166     var results = new List<Passenger, Booking>();
167
168     using var conn = getConnection();
169     conn.Open();
170
171     // combines data from the tables Booking, Passenger and Flight.
172     // The combination allows us to output data in one query.
173     using var cmd = new SqlCommand(@"
174         SELECT p.*, b.*
175         FROM Bookings b
176         JOIN Passengers p ON b.PassengerId = p.PassengerId
177         JOIN Flights f ON b.FlightId = f.FlightId
178         WHERE f.FlightNumber = @FlightNumber", conn);
179
180     cmd.Parameters.AddWithValue("@FlightNumber", flightNumber);

```

```

181 using var reader = cmd.ExecuteReader();
182
183 while (reader.Read())
184 {
185     // Combines data from Passanger and Booking into one unit of data.
186     results.Add((mapPassenger(reader), mapBooking(reader)));
187 }
188
189 return results;
190 }
191
192 public (Passenger Passenger, Booking Booking)? getPassengerBySearch(string flightNumber, string search)
193 {
194     using var conn = getConnection();
195     conn.Open();
196
197     // Search's either by PassengerId or PassportNumber.
198     using var cmd = new SqlCommand(@"
199         SELECT p.*, b.*
200         FROM Bookings b
201         JOIN Passengers p ON b.PassengerId = p.PassengerId
202         JOIN Flights f ON b.FlightId = f.FlightId
203         WHERE f.FlightNumber = @FlightNumber
204         AND (CAST(p.PassengerId AS NVARCHAR) = @Search
205             OR p.PassportNumber = @Search)", conn);
206
207     cmd.Parameters.AddWithValue("@FlightNumber", flightNumber);
208     cmd.Parameters.AddWithValue("@Search", search);
209
210     using var reader = cmd.ExecuteReader();
211
212     if (!reader.Read()) return null;
213     return (mapPassenger(reader), mapBooking(reader));
214 }
215
216 public void updateBookingStatus(int bookingId, string newStatus, DateTime? checkInTime, DateTime? boardingTime)

```

```

218 {
219     using var conn = getConnection();
220     conn.Open();
221
222     using var cmd = new SqlCommand(@"
223         UPDATE Bookings
224         SET BookingStatus = @Status,
225             CheckInTime = @CheckInTime,
226             BoardingTime = @BoardingTime
227         WHERE BookingId = @BookingId", conn);
228
229     cmd.Parameters.AddWithValue("@Status", newStatus);
230
231     // DBNull allows us to store null values within the database.
232     // This will be done till a action takes place between checkIn and boardingTime.
233     cmd.Parameters.AddWithValue("@CheckInTime", (object?)checkInTime ?? DBNull.Value);
234     cmd.Parameters.AddWithValue("@BoardingTime", (object?)boardingTime ?? DBNull.Value);
235     cmd.Parameters.AddWithValue("@BookingId", bookingId);
236
237     cmd.ExecuteNonQuery();
238 }
239
240
241 // *****
242 //             --- Aircraft Methods ---
243 // *****
244
245 public List<Aircraft> getAllAircraft()
246 {
247     var aircraft = new List<Aircraft>();
248
249     using var conn = GetConnection();
250     conn.Open();
251
252     using var cmd = new SqlCommand("SELECT * FROM Aircraft", conn);
253

```

```

254 using var reader = cmd.ExecuteReader();
255
256 while (reader.Read())
257 {
258     aircraft.Add(new Aircraft(
259         (int)reader["AircraftId"],
260         reader["AircraftType"].ToString() ?? "",
261         reader["RegistrationNumber"].ToString() ?? "",
262         reader["Manufacturer"].ToString() ?? "",
263         (int)reader["Capacity"],
264         reader["Status"].ToString() ?? ""
265     ));
266 }
267
268 return aircraft;
269 }
270
271 // *****
272 // --- Airport Methods ---
273 // *****
274
275 public List<Airport> getAllAirports()
276 {
277     var airports = new List<Airport>();
278
279     using var conn = GetConnection();
280     conn.Open();
281
282     using var cmd = new SqlCommand("SELECT * FROM Airports", conn);
283     using var reader = cmd.ExecuteReader();
284
285     while (reader.Read())
286     {
287

```

```

288     {
289         airports.Add(new Airport(
290             (int)reader["AirportId"],
291             reader["AirportCode"].ToString() ?? "",
292             reader["AirportName"].ToString() ?? "",
293             reader["City"].ToString() ?? "",
294             reader["Country"].ToString() ?? ""
295         ));
296     }
297
298     return airports;
299 }
300
301 // *****
302 // --- Crew Methods ---
303 // *****
304
305 public List<Crew> getAllCrew()
306 {
307     var crew = new List<Crew>();
308
309     using var conn = GetConnection();
310     conn.Open();
311
312     using var cmd = new SqlCommand("SELECT * FROM Crew", conn);
313     using var reader = cmd.ExecuteReader();
314
315     while (reader.Read())
316     {
317         crew.Add(new Crew(
318             (int)reader["CrewId"],
319             (int)reader["UserId"],
320             reader["CrewType"].ToString() ?? "",
321             reader["LicenseNumber"].ToString() ?? "",
322             (int)reader["YearsOfExperience"],
323             reader["Status"].ToString() ?? ""
324         ));
325     }

```

```

325         reader["Status"].ToString() ?? ""
326     });
327 }
328
329     return crew;
330 }
331
332 public void addCrew(int userId, string crewType, string licenseNumber, int yearsOfExperience)
333 {
334     using var conn = getConnection();
335     conn.Open();
336
337     using var cmd = new SqlCommand(@"
338         INSERT INTO Crew (UserId, CrewType, LicenseNumber, YearsOfExperience, Status)
339         VALUES (@UserId, @CrewType, @LicenseNumber, @YearsOfExperience, 'Active')", conn);
340
341     cmd.Parameters.AddWithValue("@UserId", userId);
342     cmd.Parameters.AddWithValue("@CrewType", crewType);
343     cmd.Parameters.AddWithValue("@LicenseNumber", licenseNumber);
344     cmd.Parameters.AddWithValue("@YearsOfExperience", yearsOfExperience);
345
346     cmd.ExecuteNonQuery();
347 }
348
349
350 // *****
351 //             --- Mapper Methods ---
352 // *****
353
354 // Converts a row within the database into a object.
355 private Flight mapFlight(SqlDataReader reader) => new Flight(
356     (int)reader["FlightId"],
357     reader["FlightNumber"].ToString() ?? "",
358     reader["Origin"].ToString() ?? "",
359     reader["Destination"].ToString() ?? "",
360     (DateTime)reader["DepartureTime"],
361     (DateTime)reader["ArrivalTime"],

```

```

362     (int)reader["Capacity"],
363     (int)reader["CurrentOccupancy"],
364     reader["Status"].ToString() ?? "",
365     (int)reader["GateAgentId"]
366 );
367
368 private Passenger mapPassenger(SqlDataReader reader) => new Passenger(
369     (int)reader["PassengerId"],
370     (int)reader["UserId"],
371     reader["PassportNumber"].ToString() ?? "",
372     (DateTime)reader["DateOfBirth"],
373     reader["Nationality"].ToString() ?? "",
374     reader["ContactNumber"].ToString() ?? ""
375 );
376
377 private Booking mapBooking(SqlDataReader reader) => new Booking(
378     (int)reader["BookingId"],
379     (int)reader["FlightId"],
380     (int)reader["PassengerId"],
381     reader["SeatNumber"].ToString() ?? "",
382     reader["BookingStatus"].ToString() ?? "",
383     (DateTime)reader["BookingDate"],
384     reader["CheckInTime"] == DBNull.Value ? null : (DateTime?)reader["CheckInTime"],
385     reader["BoardingTime"] == DBNull.Value ? null : (DateTime?)reader["BoardingTime"]
386 );
387 }
388 }

```

FlightService.cs

```
1  using SkyFlowManagement.Database;
2  using SkyFlowManagement.Model;
3  using SkyFlowManagement.Render;
4
5  namespace SkyFlowManagement.Logic
6  {
7      // FlightService handles all logic related to flights and passengers.
8      // The dashboards call methods here without direct communication to the database.
9
10     public class FlightService
11     {
12         private readonly DataHandling _repository;
13
14         public FlightService(DataHandling repository)
15         {
16             _repository = repository;
17         }
18
19         // *****
20         //      --- Admin: View All Flights ---
21         // *****
22
23         public void viewAllFlights()
24         {
25             // Gets all flights from the database.
26             List<Flight> flights = _repository.GetAllFlights();
27
28             if (flights.Count == 0)
29             {
30                 Console.WriteLine(" No flights found.");
31                 return;
32             }
33
34             // Defines the column headers of the table with the flight schedules:
35             string[] headers = { "Flight No", "Origin", "Destination", "Departure", "Arrival", "Capacity", "Occupancy", "Status" };
36
37             // Convert each flight into a string array row.
38             List<string[]> rows = flights.Select(f => new string[]
39             {
40                 f.FlightNumber,
41                 f.Origin,
42                 f.Destination,
43                 f.DepartureTime.ToString("dd/MM/yyyy HH:mm"),
44                 f.ArrivalTime.ToString("dd/MM/yyyy HH:mm"),
45                 f.Capacity.ToString(),
46                 f.CurrentOccupancy.ToString(),
47                 f.Status
48             }).ToList();
49
50             // Pass the data to the table renderer class.
51             TableCreate.Render(headers, rows);
52         }
53
54         // *****
55         //      --- Admin: Add New Flight ---
56         // *****
57
58         public void addFlight()
59         {
60             Console.Clear();
61             Console.WriteLine(" Add New Flight");
62             Console.WriteLine(" -----");
63
64             Console.Write(" Flight Number: ");
65             string flightNumber = Console.ReadLine() ?? "";
66
67             Console.Write(" Origin (e.g. CPT): ");
68             string origin = Console.ReadLine() ?? "";
69
70             Console.Write(" Destination (e.g. JHB): ");
71             string destination = Console.ReadLine() ?? "";
72         }
73     }
74 }
```

```

72 string destination = Console.ReadLine() ?? "";
73
74 // Loops until a valid date is entered.
75 DateTime departureTime;
76 Console.Write(" Departure Time (dd/MM/yyyy HH:mm): ");
77 while (!DateTime.TryParseExact(Console.ReadLine(), "dd/MM/yyyy HH:mm",
78 null, System.Globalization.DateTimeStyles.None, out departureTime))
79 {
80     Console.Write(" Invalid format. Try again (dd/MM/yyyy HH:mm): ");
81 }
82
83 DateTime arrivalTime;
84 Console.Write(" Arrival Time (dd/MM/yyyy HH:mm): ");
85 while (!DateTime.TryParseExact(Console.ReadLine(), "dd/MM/yyyy HH:mm",
86 null, System.Globalization.DateTimeStyles.None, out arrivalTime))
87 {
88     Console.Write(" Invalid format. Try again (dd/MM/yyyy HH:mm): ");
89 }
90
91 // Loops until a valid number is entered.
92 int capacity;
93 Console.Write(" Capacity: ");
94 while (!int.TryParse(Console.ReadLine(), out capacity) || capacity <= 0)
95 {
96     Console.Write(" Invalid. Enter a number greater than 0: ");
97 }
98
99 int gateAgentId;
100 Console.Write(" Gate Agent ID: ");
101 while (!int.TryParse(Console.ReadLine(), out gateAgentId) || gateAgentId <= 0)
102 {
103     Console.Write(" Invalid. Enter a valid Gate Agent ID: ");
104 }
105
106 // Once all inputs are validated and collected,
107 // The are then stored into the database.
108 _repository.AddFlight(flightNumber, origin, destination, departureTime, arrivalTime, capacity, gateAgentId);
109 Console.WriteLine("\n Flight added successfully.");
110 }
111
112 // *****
113 // --- Admin: Add Staff ---
114 // *****
115
116 public void addStaff()
117 {
118     Console.Clear();
119     Console.WriteLine(" Add New Staff Member");
120     Console.WriteLine(" -----");
121
122     Console.Write(" Username: ");
123     string username = Console.ReadLine() ?? "";
124
125     Console.Write(" Password: ");
126     string password = Console.ReadLine() ?? "";
127     string passwordHash = AuthService.HashPassword(password);
128
129     Console.Write(" Role (Admin / GateAgent): ");
130     string role = Console.ReadLine() ?? "";
131     while (role != "Admin" && role != "GateAgent")
132     {
133         Console.Write(" Invalid role. Enter Admin or GateAgent: ");
134         role = Console.ReadLine() ?? "";
135     }
136
137     Console.Write(" Email: ");
138     string email = Console.ReadLine() ?? "";
139
140     Console.Write(" First Name: ");
141     string firstName = Console.ReadLine() ?? "";
142
143     Console.Write(" Last Name: ");

```

```

145     string lastName = Console.ReadLine() ?? "";
146
147     _repository.addUser(username, passwordHash, role, email, firstName, lastName);
148     Console.WriteLine("\n Staff member added successfully.");
149 }
150
151
152 // *****
153 //      --- Gate Agent: View Flight Manifest ---
154 // *****
155
156 public void viewFlightManifest(string flightNumber)
157 {
158     // First check if the specific flight exists.
159     Flight? flight = _repository.GetFlightByNumber(flightNumber);
160     if (flight == null)
161     {
162         Console.WriteLine($" Flight {flightNumber} not found.");
163         return;
164     }
165
166     // Get all passengers on the flight mentioned above.
167     List<Passenger Passenger, Booking Booking> manifest =
168         _repository.GetPassengersByFlightNumber(flightNumber);
169
170     if (manifest.Count == 0)
171     {
172         Console.WriteLine(" No passengers found for this flight.");
173         return;
174     }
175
176     Console.WriteLine($" \n Flight {flightNumber} - {flight.Origin} to {flight.Destination}");
177     Console.WriteLine($" Status: {flight.Status} | Occupancy: {flight.CurrentOccupancy}/{flight.Capacity}\n");
178
179     string[] headers = { "Passenger ID", "Passport", "Nationality", "Seat", "Status" };
180
181     List<string[]> rows = manifest.Select(x => new string[]
182     {
183         x.Passenger.PassengerId.ToString(),
184         x.Passenger.PassportNumber,
185         x.Passenger.Nationality,
186         x.Booking.SeatNumber,
187         x.Booking.BookingStatus
188     }).ToList();
189
190     TableCreate.Render(headers, rows);
191 }
192
193
194 // *****
195 //      --- Gate Agent: Check In Passenger ---
196 // *****
197
198 public void CheckInPassenger(string flightNumber, string search)
199 {
200     // Rule 1: The flight must exist.
201     Flight? flight = _repository.getFlightByNumber(flightNumber);
202     if (flight == null)
203     {
204         Console.WriteLine($" Flight {flightNumber} not found.");
205         return;
206     }
207
208     // Rule 2: Passangers can't check in on a departed flight.
209     if (flight.Status == "Departed")
210     {
211         Console.WriteLine(" Cannot check in a passenger - this flight has already departed.");
212         return;
213     }
214
215     // Rule 3: Check for the specific passenger.
216     var result = _repository.getPassengerBySearch(flightNumber, search);
217     if (result == null)
218     {

```

```

219         Console.WriteLine(" Passenger not found.");
220         return;
221     }
222
223     var (passenger, booking) = result.Value;
224
225     // Rule 4: if the passanger exists
226     // checks if passanger already checked in.
227     if (booking.bookingStatus == "CheckedIn")
228     {
229         Console.WriteLine($" {passenger.PassportNumber} is already checked in (Seat {booking.SeatNumber}).");
230         return;
231     }
232
233     // Rule 5: checks if passanger already boarded.
234     if (booking.bookingStatus == "Boarded")
235     {
236         Console.WriteLine($" {passenger.PassportNumber} has already boarded.");
237         return;
238     }
239
240     // Once all rules are followed and checked
241     // Passanger booking status is updated.
242     booking.CheckIn();
243     _repository.updateBookingStatus(booking.BookingId, booking.BookingStatus, booking.CheckInTime, booking.BoardingTime);
244
245     Console.WriteLine($" \n Passenger found: {passenger.PassportNumber} (Seat {booking.SeatNumber})");
246     Console.WriteLine($" Status updated to: CheckedIn");
247 }
248
249
250 // *****
251 // --- Gate Agent: Board Passenger ---
252 // *****
253
254 public void BoardPassenger(string flightNumber, string search)
255 {
256     // Rule:1: The flight must exist.
257     Flight? flight = _repository.getFlightByNumber(flightNumber);
258     if (flight == null)
259     {
260         Console.WriteLine($" Flight {flightNumber} not found.");
261         return;
262     }
263
264     // Rule 2: Passanger can't board on a departed flight.
265     if (flight.Status == "Departed")
266     {
267         Console.WriteLine(" Cannot board a passenger - this flight has already departed.");
268         return;
269     }
270
271     // Rule 3: Passanger cannot board if flight is full.
272     if (flight.IsFull())
273     {
274         Console.WriteLine(" Cannot board - this flight is at full capacity.");
275         return;
276     }
277
278     // Checks for the specific passenger.
279     var result = _repository.GetPassengerBySearch(flightNumber, search);
280     if (result == null)
281     {
282         Console.WriteLine(" Passenger not found.");
283         return;
284     }
285
286     var (passenger, booking) = result.Value;
287
288     // Rule 4: Passangers must be checked in before boarding.
289     if (booking.BookingStatus != "CheckedIn")
290     {
291         Console.WriteLine($" Passenger must be checked in before boarding. Current status: {booking.BookingStatus}");
292         return;

```

```

293     }
294
295     // Confirmation information for user before boarding.
296     Console.WriteLine($" \n Passenger found: {passenger.PassportNumber} (Seat {booking.SeatNumber})");
297     Console.WriteLine($" Current status: {booking.BookingStatus}");
298     Console.Write(" Update status to Boarded? (Y/N): ");
299     string confirm = Console.ReadLine() ?? "";
300
301     if (confirm.ToUpper() != "Y")
302     {
303         Console.WriteLine(" Boarding cancelled.");
304         return;
305     }
306
307     // Once all rules are followed and checked
308     // Passanger booking status is updated.
309     // Passanger can then board the aircraft.
310     booking.Board();
311     _repository.updateBookingStatus(booking.BookingId, booking.BookingStatus, booking.CheckInTime, booking.BoardingTime);
312
313     Console.WriteLine(" Status updated successfully.");
314 }
315
316
317 // *****
318 // --- Gate Agent: Update Flight Status ---
319 // *****
320
321 public void updateFlightStatus(string flightNumber)
322 {
323     Flight? flight = _repository.getFlightByNumber(flightNumber);
324     if (flight == null)
325     {
326         Console.WriteLine($" Flight {flightNumber} not found.");
327         return;
328     }
329
330     Console.WriteLine($" Current status: {flight.Status}");
331     Console.WriteLine(" New status options: Scheduled / Boarding / Departed / Delayed / Cancelled");
332     Console.Write(" Enter new status: ");
333     string newStatus = Console.ReadLine() ?? "";
334
335     string[] validStatuses = { "Scheduled", "Boarding", "Departed", "Delayed", "Cancelled" };
336     if (!validStatuses.Contains(newStatus))
337     {
338         Console.WriteLine(" Invalid status entered.");
339         return;
340     }
341
342     // If the flight is leaving, call the departFlight method.
343     // else update the status of the flight.
344     if (newStatus == "Departed")
345         flight.departFlight();
346     else
347         flight.updateStatus(newStatus);
348
349     _repository.updateFlightStatus(flightNumber, newStatus);
350     Console.WriteLine($" Flight status updated to: {newStatus}");
351 }
352 }
353

```

AuthService.cs

```
1  using System;
2  using SkyFlowManagement.Database;
3  using SkyFlowManagement.Model;
4
5  namespace SkyFlowManagement.Logic
6  {
7      // AuthService handles the validation
8      // It sits between the UI and the database.
9
10     public class AuthService
11     {
12         // Communicates to the database through the classes within the UI repository
13         private readonly DataHandling _repository;
14
15         public AuthService(DataHandling repository)
16         {
17             _repository = repository;
18         }
19
20         // Validates user credentials from the login screen.
21         public User? Login(string username, string password)
22         {
23             // Step 1: Checks if the username exists in the database.
24             User? user = _repository.GetUserByUsername(username);
25
26             // Step 2: If the username isn't found then the program stops.
27             if (user == null) return null;
28
29             // Step 3: Hash the password the user types and compares it
30             // to the hash stored in the database.
31             string hashedInput = HashPassword(password);
32
33             // Step 4: It references and uses validatePassword() method from the User class.
34             // If they don't match, it return null, therefore the login failed
35             if (!user.validatePassword(hashedInput)) return null;
36
37             // Step 5: If all credentials are valid
38             // returns and call the user object.
39             return user;
40         }
41
42         // Converts a plain-text password into a hash.
43         // This must match exactly what LoginScreen.cs uses, and what's stored in the database.
44         public static string HashPassword(string password)
45         {
46             using var sha256 = System.Security.Cryptography.SHA256.Create();
47             byte[] bytes = sha256.ComputeHash(System.Text.Encoding.UTF8.GetBytes(password));
48             return Convert.ToBase64String(bytes);
49         }
50     }
51 }
```

TableCreate.cs

```
1 namespace SkyFlowManagement.Render
2 {
3     /* Class that specifically created to implement any class with a header and row
4        into a uniformed table to display information: E.g.: */
5     /* +-----+-----+-----+-----+
6        | Flight No | Origin | Destination | Status |
7        +-----+-----+-----+-----+
8        | SF102    | CPT   | JHB        | Boarding |
9        +-----+-----+-----+-----+
10    */
11
12    public static class TableCreate
13    {
14
15        public static void Render(string[] headers, List<string[]> rows)
16        {
17            if (headers.Length == 0)
18            {
19                Console.WriteLine(" No headers provided.");
20                return;
21            }
22
23            // Step 1: Estimates how wide the table columns need to be.
24            // Starts with the heeader with and compares with row width to see which is wider.
25            int[] columnWidths = getColumnWidths(headers, rows);
26
27            // Step 2: Builds the lines in between the rows and columns
28            string divider = buildDivider(columnWidths);
29
30            // Step 3: Prints the table as a whole.
31            Console.WriteLine();
32            Console.WriteLine(divider);
33            Console.WriteLine(buildRow(headers, columnWidths));
34            Console.WriteLine(divider);
35
36            if (rows.Count == 0)
37            {
38                // Print a single row the wide of of the table,
39                // to display if a record is not found.
40                string emptyMessage = " No records found.";
41                Console.WriteLine("| " + emptyMessage.padRight(divider.Length - 4) + " |");
42            }
43            else
44            {
45                foreach (string[] row in rows)
46                {
47                    Console.WriteLine(buildRow(row, columnWidths));
48                }
49            }
50
51            Console.WriteLine(divider);
52            Console.WriteLine($" {rows.Count} record(s) found.");
53            Console.WriteLine();
54        }
55
56        // Works out the maximum width needed for each column.
57        // Checks both the header and every data row.
58        private static int[] getColumnWidths(string[] headers, List<string[]> rows)
59        {
60            // Start with the header widths.
61            int[] widths = headers.Select(h => h.Length).ToArray();
62
63            // Checks each row and if the cell is wider, then the width is updated.
64            foreach (string[] row in rows)
65            {
66                for (int i = 0; i < row.Length && i < widths.Length; i++)
67                {
68                    int cellWidth = (row[i] ?? "").Length;
69                    if (cellWidth > widths[i])
70                    {
71                        widths[i] = cellWidth;
72                    }
73                }
74            }
75        }
76    }
77 }
```

```

75     }
76
77     // Adds to every column for padding
78     return widths.Select(w => w + 2).ToArray();
79 }
80
81
82 // Builds a divider lines.
83 private static string buildDivider(int[] columnWidths)
84 {
85     // Creates a line in the table for each column.
86     // Then joins it with a separator.
87     string[] segments = columnWidths.Select(w => new string('-', w)).ToArray();
88     return "+" + string.Join("+", segments) + "+";
89 }
90
91
92 // Builds a single data row:
93 private static string buildRow(string[] cells, int[] columnWidths)
94 {
95     string row = "|";
96
97     for (int i = 0; i < columnWidths.Length; i++)
98     {
99         // Uses an empty string if the row is shorter than what was estimated.
100         string cell = i < cells.Length ? (cells[i] ?? "") : "";
101
102         // Fills the cell with enough spaces to reach the column width.
103         // Reduce padding to leave room for the space on each side of the cell.
104         string paddedCell = " " + cell.PadRight(columnWidths[i] - 1);
105
106         row += paddedCell + "|";
107     }
108
109     return row;
110 }
111 }
112 }
113

```

LoginScreen.cs

```
1  using SkyFlowManagement.Database;
2  using SkyFlowManagement.Logic;
3  using SkyFlowManagement.Model;
4
5  namespace SkyFlowManagement.UI
6  {
7      public class LoginScreen
8      {
9          private readonly DataHandling _repository;
10
11         public LoginScreen(DataHandling repository)
12         {
13             _repository = repository;
14         }
15
16         public User? Show()
17         {
18             Console.Clear();
19             Console.WriteLine("+-----+");
20             Console.WriteLine("|   SkyFlow Terminal Manager   |");
21             Console.WriteLine("+-----+");
22
23             Console.Write("\n Enter username: ");
24             string username = Console.ReadLine() ?? "";
25
26             Console.Write(" Enter password: ");
27             string password = ReadHiddenInput();
28
29             // AuthService handles the login logic.
30             AuthService auth = new AuthService(_repository);
31             User? user = auth.Login(username, password);
32
33             if (user == null)
34             {
35                 Console.WriteLine("\n Invalid username or password. Press any key to try again.");
36                 Console.ReadKey();
37                 return null;
38             }
39
40             Console.WriteLine($"{user.Role} Authentication successful. Role: {user.Role}");
41             Thread.Sleep(1000);
42             return user;
43         }
44
45         // Reads password input
46         // And displays default symbols than showing the passwords as its typed.
47         private string ReadHiddenInput()
48         {
49             string input = "";
50             ConsoleKeyInfo key;
51
52             do
53             {
54                 key = Console.ReadKey(true);
55
56                 if (key.Key != ConsoleKey.Backspace && key.Key != ConsoleKey.Enter)
57                 {
58                     input += key.KeyChar;
59                     Console.Write("*");
60                 }
61                 else if (key.Key == ConsoleKey.Backspace && input.Length > 0)
62                 {
63                     // Remove the last symbol from password input and erase the * on screen.
64                     input = input[..^1];
65                     Console.Write("\b \b");
66                 }
67             }
68             while (key.Key != ConsoleKey.Enter);
69
70             Console.WriteLine();
71             return input;
72         }
73     }
74 }
```

AdminDashboard.cs

```
1  using SkyFlowManagement.Database;
2  using SkyFlowManagement.Logic;
3  using SkyFlowManagement.Model;
4
5  namespace SkyFlowManagement.UI
6  {
7      public class AdminDashBoard
8      {
9          private readonly Admin _admin;
10         private readonly FlightService _flightService;
11
12         public AdminDashBoard(Admin admin, DataHandling repository)
13         {
14             _admin = admin;
15             // AdminDashboard uses the FlightService class.
16             // This allows this class to not have to communicate to the database directly.
17             _flightService = new FlightService(repository);
18         }
19
20         public void Show()
21         {
22             bool running = true;
23
24             while (running)
25             {
26                 Console.Clear();
27
28                 // Polymorphism: calls the method from the Admin sub-class
29                 // It therefore overrides the implementation.
30                 _admin.DisplayDashboard();
31
32                 Console.Write("\n Select option: ");
33                 string input = Console.ReadLine() ?? "";
34
35                 switch (input.Trim())
36                 {
37                     case "1":
38                         manageFlights();
39                         break;
40                     case "2":
41                         viewSystemOverview();
42                         break;
43                     case "3":
44                         manageStaff();
45                         break;
46                     case "0":
47                         running = false;
48                         break;
49                     default:
50                         Console.WriteLine("\n Invalid option. Press any key to try again.");
51                         Console.ReadKey();
52                         break;
53                 }
54             }
55         }
56
57         private void manageFlights()
58         {
59             Console.Clear();
60             Console.WriteLine(" Manage Flights");
61             Console.WriteLine(" -----");
62             Console.WriteLine(" 1. Add New Flight");
63             Console.WriteLine(" 2. View All Flights");
64             Console.WriteLine(" 3. Update Flight Status");
65             Console.Write("\n Select option: ");
66             string input = Console.ReadLine() ?? "";
67
68             switch (input.Trim())
69             {
70                 case "1":
71                     _flightService.addFlight();
72                     break;
73                 case "2":
74                     _flightService.viewAllFlights();
```

```

75         break;
76     case "3":
77         Console.WriteLine(" Enter Flight Number: ");
78         string flightNumber = Console.ReadLine() ?? "";
79         _flightService.updateFlightStatus(flightNumber);
80         break;
81     default:
82         Console.WriteLine(" Invalid option.");
83         break;
84     }
85
86     Console.WriteLine("\n Press any key to continue.");
87     Console.ReadKey();
88 }
89
90 private void viewSystemOverview()
91 {
92     Console.Clear();
93     Console.WriteLine(" System Overview - All Flights");
94     Console.WriteLine(" -----");
95     _flightService.viewAllFlights();
96     Console.WriteLine("\n Press any key to continue.");
97     Console.ReadKey();
98 }
99
100 private void manageStaff()
101 {
102     Console.Clear();
103     Console.WriteLine(" Manage Staff");
104     Console.WriteLine(" -----");
105     Console.WriteLine(" 1. Add New Staff Member");
106     Console.Write("\n Select option: ");
107     string input = Console.ReadLine() ?? "";
108
109     switch (input.Trim())
110     {
111     case "1":
112         _flightService.AddStaff();
113         break;
114     default:
115         Console.WriteLine(" Invalid option.");
116         break;
117     }
118
119     Console.WriteLine("\n Press any key to continue.");
120     Console.ReadKey();
121 }
122 }
123

```

GateAgentDashBaord.cs

```
1  using SkyFlowManagement.Database;
2  using SkyFlowManagement.Logic;
3  using SkyFlowManagement.Model;
4
5  namespace SkyFlowManagement.UI
6  {
7      public class GateAgentDashboard
8      {
9          private readonly GateAgent _gateAgent;
10         private readonly FlightService _flightService;
11
12         public GateAgentDashboard(GateAgent gateAgent, DataHandling repository)
13         {
14             _gateAgent = gateAgent;
15             _flightService = new FlightService(repository);
16         }
17
18         public void Show()
19         {
20             bool running = true;
21
22             while (running)
23             {
24                 Console.Clear();
25
26                 // Polymorphism: GateAgentDashboard uses the FlightService class.
27                 // This allows this class to not have to communicate to the database directly.
28                 _gateAgent.DisplayDashboard();
29
30                 Console.WriteLine("\n Select option: ");
31                 string input = Console.ReadLine() ?? "";
32
33                 switch (input.Trim())
34                 {
35                     case "1":
36                         ViewFlightManifest();
37                         break;
38                     case "2":
39                         PassengerCheckIn();
40                         break;
41                     case "3":
42                         BoardingGate();
43                         break;
44                     case "0":
45                         running = false;
46                         break;
47                     default:
48                         Console.WriteLine("\n Invalid option. Press any key to try again.");
49                         Console.ReadKey();
50                         break;
51                 }
52             }
53         }
54
55         private void ViewFlightManifest()
56         {
57             Console.Clear();
58             Console.WriteLine(" Flight Manifest");
59             Console.WriteLine(" -----");
60             Console.Write(" Enter Flight Number: ");
61             string flightNumber = Console.ReadLine() ?? "";
62
63             _flightService.ViewFlightManifest(flightNumber);
64
65             Console.WriteLine("\n Press any key to continue.");
66             Console.ReadKey();
67         }
68
69         private void PassengerCheckIn()
70         {
71             Console.Clear();
72             Console.WriteLine(" Passenger Check-In");
73             Console.WriteLine(" -----");
74             Console.Write(" Enter Flight Number: ");
```

```

75     string flightNumber = Console.ReadLine() ?? "";
76
77     Console.Write(" Enter Passenger ID or Passport Number: ");
78     string search = Console.ReadLine() ?? "";
79
80     _flightService.CheckInPassenger(flightNumber, search);
81
82     Console.WriteLine("\n Press any key to continue.");
83     Console.ReadKey();
84 }
85
86 private void BoardingGate()
87 {
88     Console.Clear();
89     Console.WriteLine(" Boarding Gate");
90     Console.WriteLine(" -----");
91     Console.Write(" Enter Flight Number: ");
92     string flightNumber = Console.ReadLine() ?? "";
93
94     Console.Write(" Enter Passenger ID or Passport Number: ");
95     string search = Console.ReadLine() ?? "";
96
97     _flightService.BoardPassenger(flightNumber, search);
98
99     Console.WriteLine("\n Press any key to continue.");
100    Console.ReadKey();
101 }
102 }
103 }

```

Program.cs

```
1  using SkyFlowManagement.Model;
2  using SkyFlowManagement.Logic;
3  using SkyFlowManagement.Database;
4  using SkyFlowManagement.UI;
5
6  namespace SkyFlowManagement
7  {
8      internal class Program
9      {
10         static void Main()
11         {
12             string connectionString = "Server=(localdb)\\MSSQLLocalDB;Database=dbSkyFlow;Trusted_Connection=True;TrustServerCertificate=True;";
13
14             DataHandling repository = new SQLRepo(connectionString);
15
16             LoginScreen loginScreen = new LoginScreen(repository);
17
18             bool appRunning = true;
19
20             while (appRunning)
21             {
22                 User? user = loginScreen.Show();
23
24                 if (user == null) continue;
25
26                 if (user is Admin admin)
27                 {
28                     AdminDashBoard adminDashboard = new AdminDashBoard(admin, repository);
29                     adminDashboard.Show();
30                 }
31                 else if (user is GateAgent gateAgent)
32                 {
33                     GateAgentDashboard gateAgentDashboard = new GateAgentDashboard(gateAgent, repository);
34                     gateAgentDashboard.Show();
35                 }
36
37                 Console.Clear();
38                 Console.WriteLine(" You have been logged out.");
39                 Console.WriteLine(" Press any key to return to the login screen, or close the window to exit.");
40                 Console.ReadKey();
41             }
42         }
43     }
44 }
```

CreateTable.sql

```
1
2  -- It creates the SkyFlowDB database and all the tables.
3
4  -- Step 1: Create the database if it doesn't exist.
5  IF NOT EXISTS (SELECT name FROM sys.databases WHERE name = 'dbSkyFlow')
6  BEGIN
7      CREATE DATABASE dbSkyFlow;
8  END
9  GO
10
11  USE dbSkyFlow;
12  GO
13
14
15  -- Users table: stores all staff (Admins and Gate Agents).
16  IF NOT EXISTS (SELECT * FROM sys.tables WHERE name = 'Users')
17  BEGIN
18  CREATE TABLE Users (
19      UserId          INT          PRIMARY KEY IDENTITY(1,1),
20      Username        NVARCHAR(50) NOT NULL UNIQUE,
21      PasswordHash    NVARCHAR(255) NOT NULL,
22      Role            NVARCHAR(20) NOT NULL,
23      Email           NVARCHAR(100) NOT NULL,
24      FirstName       NVARCHAR(50) NOT NULL,
25      LastName        NVARCHAR(50) NOT NULL,
26      CreatedAt       DATETIME     NOT NULL DEFAULT GETDATE()
27  );
28  END
29  GO
30
31  -- Aircraft table: stores aircraft details.
32  IF NOT EXISTS (SELECT * FROM sys.tables WHERE name = 'Aircraft')
33  BEGIN
34  CREATE TABLE Aircraft (
35      AircraftId      INT          PRIMARY KEY IDENTITY(1,1),
36      AircraftType     NVARCHAR(50) NOT NULL,
37      RegistrationNumber NVARCHAR(20) NOT NULL UNIQUE,
38      Manufacturer     NVARCHAR(50) NOT NULL,
39      Capacity        INT          NOT NULL,
40      Status           NVARCHAR(20) NOT NULL DEFAULT 'Active'
41  );
42  END
43  GO
44
45  -- Airports table: stores airport details.
46  IF NOT EXISTS (SELECT * FROM sys.tables WHERE name = 'Airports')
47  BEGIN
48  CREATE TABLE Airports (
49      AirportId       INT          PRIMARY KEY IDENTITY(1,1),
50      AirportCode      NVARCHAR(10) NOT NULL UNIQUE,
51      AirportName      NVARCHAR(100) NOT NULL,
52      City            NVARCHAR(50) NOT NULL,
53      Country          NVARCHAR(50) NOT NULL
54  );
55  END
56  GO
57
58
59  -- Flights table: depends on the Users
60  IF NOT EXISTS (SELECT * FROM sys.tables WHERE name = 'Flights')
61  BEGIN
62  CREATE TABLE Flights (
63      FlightId        INT          PRIMARY KEY IDENTITY(1,1),
64      FlightNumber     NVARCHAR(10) NOT NULL UNIQUE,
65      Origin           NVARCHAR(50) NOT NULL,
66      Destination      NVARCHAR(50) NOT NULL,
67      DepartureTime    DATETIME     NOT NULL,
68      ArrivalTime      DATETIME     NOT NULL,
```

```

69         Capacity          INT          NOT NULL,
70         CurrentOccupancy  INT          NOT NULL DEFAULT 0,
71         Status            NVARCHAR(20)  NOT NULL DEFAULT 'Scheduled',
72         GateAgentId       INT          NOT NULL,
73         FOREIGN KEY (GateAgentId) REFERENCES Users(UserId)
74     );
75 END
76 GO
77
78 -- Passengers table: depends on Users
79 < IF NOT EXISTS (SELECT * FROM sys.tables WHERE name = 'Passengers')
80 < BEGIN
81 <     CREATE TABLE Passengers (
82         PassengerId      INT          PRIMARY KEY IDENTITY(1,1),
83         UserId           INT          NOT NULL,
84         PassportNumber   NVARCHAR(20) NOT NULL UNIQUE,
85         DateOfBirth      DATE        NOT NULL,
86         Nationality      NVARCHAR(50) NOT NULL,
87         ContactNumber    NVARCHAR(20) NOT NULL,
88         FOREIGN KEY (UserId) REFERENCES Users(UserId)
89     );
90 END
91 GO
92
93 -- Crew table: depends on Users (UserId).
94 < IF NOT EXISTS (SELECT * FROM sys.tables WHERE name = 'Crew')
95 < BEGIN
96 <     CREATE TABLE Crew (
97         CrewId           INT          PRIMARY KEY IDENTITY(1,1),
98         UserId           INT          NOT NULL,
99         CrewType         NVARCHAR(30) NOT NULL,
100        LicenseNumber    NVARCHAR(50) NOT NULL UNIQUE,
101        YearsOfExperience INT          NOT NULL,
102        Status           NVARCHAR(20) NOT NULL DEFAULT 'Active',
103        FOREIGN KEY (UserId) REFERENCES Users(UserId)
104    );
105 END
106 GO
107
108 -- Bookings table: depends on Flights and Passengers.
109 < IF NOT EXISTS (SELECT * FROM sys.tables WHERE name = 'Bookings')
110 < BEGIN
111 <     CREATE TABLE Bookings (
112         BookingId        INT          PRIMARY KEY IDENTITY(1,1),
113         FlightId         INT          NOT NULL,
114         PassengerId      INT          NOT NULL,
115         SeatNumber       NVARCHAR(10) NOT NULL,
116         BookingStatus    NVARCHAR(20) NOT NULL DEFAULT 'Confirmed',
117         BookingDate      DATETIME    NOT NULL DEFAULT GETDATE(),
118         CheckInTime      DATETIME    NULL,
119         BoardingTime     DATETIME    NULL,
120         FOREIGN KEY (FlightId) REFERENCES Flights(FlightId),
121         FOREIGN KEY (PassengerId) REFERENCES Passengers(PassengerId)
122     );
123 END
124 GO
125
126 -- FlightAssignment table: depends on Flights, Aircraft, and Users.
127 < IF NOT EXISTS (SELECT * FROM sys.tables WHERE name = 'FlightAssignment')
128 < BEGIN
129 <     CREATE TABLE FlightAssignment (
130         AssignmentId     INT          PRIMARY KEY IDENTITY(1,1),
131         FlightId         INT          NOT NULL,
132         AircraftId       INT          NOT NULL,
133         PilotId          INT          NOT NULL,
134         CoPilotId        INT          NOT NULL,
135         AssignedDate     DATETIME    NOT NULL DEFAULT GETDATE(),
136         FOREIGN KEY (FlightId) REFERENCES Flights(FlightId),

```

```

137         FOREIGN KEY (AircraftId) REFERENCES Aircraft(AircraftId),
138         FOREIGN KEY (PilotId) REFERENCES Users(UserId),
139         FOREIGN KEY (CoPilotId) REFERENCES Users(UserId)
140     );
141 END
142 GO
143
144 -- Baggage table: depends on Bookings.
145 < IF NOT EXISTS (SELECT * FROM sys.tables WHERE name = 'Baggage')
146 < BEGIN
147 <     CREATE TABLE Baggage (
148 <         BaggageId INT PRIMARY KEY IDENTITY(1,1),
149 <         BookingId INT NOT NULL,
150 <         Weight DECIMAL(5,2) NOT NULL,
151 <         BaggageTag NVARCHAR(20) NOT NULL UNIQUE,
152 <         Status NVARCHAR(20) NOT NULL DEFAULT 'Checked',
153 <         FOREIGN KEY (BookingId) REFERENCES Bookings(BookingId)
154 <     );
155 END
156 GO
157
158 -- FlightLog table: depends on Flights and Users.
159 < IF NOT EXISTS (SELECT * FROM sys.tables WHERE name = 'FlightLog')
160 < BEGIN
161 <     CREATE TABLE FlightLog (
162 <         LogId INT PRIMARY KEY IDENTITY(1,1),
163 <         FlightId INT NOT NULL,
164 <         Action NVARCHAR(50) NOT NULL,
165 <         PerformedBy INT NOT NULL,
166 <         PerformedAt DATETIME NOT NULL DEFAULT GETDATE(),
167 <         Details NVARCHAR(500) NULL,
168 <         FOREIGN KEY (FlightId) REFERENCES Flights(FlightId),
169 <         FOREIGN KEY (PerformedBy) REFERENCES Users(UserId)
170 <     );
171 END
172 GO
173
174 -- Notification table: depends on Users.
175 IF NOT EXISTS (SELECT * FROM sys.tables WHERE name = 'Notifications')
176 BEGIN
177     CREATE TABLE Notifications (
178         NotificationId INT PRIMARY KEY IDENTITY(1,1),
179         UserId INT NOT NULL,
180         Message NVARCHAR(500) NOT NULL,
181         Type NVARCHAR(30) NOT NULL,
182         IsRead BIT NOT NULL DEFAULT 0,
183         CreatedAt DATETIME NOT NULL DEFAULT GETDATE(),
184         FOREIGN KEY (UserId) REFERENCES Users(UserId)
185     );
186 END
187 GO
188
189 -- AuditLog table: depends on Users.
190 IF NOT EXISTS (SELECT * FROM sys.tables WHERE name = 'AuditLog')
191 BEGIN
192     CREATE TABLE AuditLog (
193         AuditId INT PRIMARY KEY IDENTITY(1,1),
194         UserId INT NOT NULL,
195         Action NVARCHAR(100) NOT NULL,
196         TableName NVARCHAR(50) NOT NULL,
197         RecordId INT NOT NULL,
198         OldValues NVARCHAR(MAX) NULL,
199         NewValues NVARCHAR(MAX) NULL,
200         Timestamp DATETIME NOT NULL DEFAULT GETDATE(),
201         FOREIGN KEY (UserId) REFERENCES Users(UserId)
202     );
203 END
204 GO

```

seedData.sql

```
1  -- Inserts Test data that will be used to run the App.
2
3  USE dbSkyFlow;
4  GO
5
6  -- Clear existing data first to avoid duplicates.
7  DELETE FROM AuditLog;
8  DELETE FROM Notifications;
9  DELETE FROM FlightLog;
10 DELETE FROM Baggage;
11 DELETE FROM FlightAssignment;
12 DELETE FROM Bookings;
13 DELETE FROM Crew;
14 DELETE FROM Passengers;
15 DELETE FROM Flights;
16 DELETE FROM Aircraft;
17 DELETE FROM Airports;
18 DELETE FROM Users;
19 GO
20
21 -- Reset identity counters after delete.
22 DBCC CHECKIDENT ('Users', RESEED, 0);
23 DBCC CHECKIDENT ('Flights', RESEED, 0);
24 DBCC CHECKIDENT ('Passengers', RESEED, 0);
25 DBCC CHECKIDENT ('Bookings', RESEED, 0);
26 DBCC CHECKIDENT ('Aircraft', RESEED, 0);
27 DBCC CHECKIDENT ('Airports', RESEED, 0);
28 DBCC CHECKIDENT ('Crew', RESEED, 0);
29 GO
30
31 -- *****
32 --      - Users table -
33 -- *****
34 -- Passwords are SHA256 hashed then Base64 encoded - matching AuthService.HashPassword().
35 -- admin123 = jG125bVBBBW96QI9Te4V37Fnqchz/Eu4qB9vKrRIqRg=
36 -- agent123 = 2duLHGyZhtDjQBSMbNqGhFcFNMgAJMLH500XBT50VHI=
37 -- Pass123 = iNQmb9tcolUrQuxFVHGBjX/346pc7bFTBY7T0NZCKZU=
38
39 INSERT INTO Users (Username, PasswordHash, Role, Email, FirstName, LastName)
40 VALUES
41 ('admin', 'jG125bVBBBW96QI9Te4V37Fnqchz/Eu4qB9vKrRIqRg=', 'Admin', 'admin@skyflow.com', 'Nara', 'Smith'),
42 ('agent1', '2duLHGyZhtDjQBSMbNqGhFcFNMgAJMLH500XBT50VHI=', 'GateAgent', 'agent1@skyflow.com', 'Karabo', 'Dlamini'),
43 ('agent2', '2duLHGyZhtDjQBSMbNqGhFcFNMgAJMLH500XBT50VHI=', 'GateAgent', 'agent2@skyflow.com', 'Michael', 'Jordan'),
44 ('pas1', 'iNQmb9tcolUrQuxFVHGBjX/346pc7bFTBY7T0NZCKZU=', 'GateAgent', 'pas1@email.com', 'Solomon', 'Smith'),
45 ('pas2', 'iNQmb9tcolUrQuxFVHGBjX/346pc7bFTBY7T0NZCKZU=', 'GateAgent', 'pas2@email.com', 'John', 'Doe');
46 GO
47
48 -- *****
49 --      - Aircraft table -
50 -- *****
51 INSERT INTO Aircraft (AircraftType, RegistrationNumber, Manufacturer, Capacity, Status)
52 VALUES
53 ('Boeing 737', 'ZS-ABC', 'Boeing', 150, 'Active'),
54 ('Airbus A320', 'ZS-DEF', 'Airbus', 120, 'Active'),
55 ('Boeing 787', 'ZS-GHI', 'Boeing', 200, 'Active');
56 GO
57
58 -- *****
59 --      - Airports table -
60 -- *****
61 INSERT INTO Airports (AirportCode, AirportName, City, Country)
62 VALUES
63 ('CPT', 'Cape Town International', 'Cape Town', 'South Africa'),
64 ('JHB', 'OR Tambo International', 'Johannesburg', 'South Africa'),
65 ('DBN', 'King Shaka International', 'Durban', 'South Africa'),
66 ('PLZ', 'Chief Dawid Stuurman International', 'Port Elizabeth', 'South Africa');
67 GO
68 |
```

```

69  -- *****
70  --      - Flights table -
71  -- *****
72
73  -- GateAgentId 2 = agent1, GateAgentId 3 = agent2
74
75  INSERT INTO Flights (FlightNumber, Origin, Destination, DepartureTime, ArrivalTime, Capacity, CurrentOccupancy, Status, GateAgentId)
76  VALUES
77  ('SF102', 'CPT', 'JHB', DATEADD(hour, 2, GETDATE()), DATEADD(hour, 4, GETDATE()), 150, 2, 'Boarding', 2),
78  ('SF221', 'DBN', 'CPT', DATEADD(hour, 4, GETDATE()), DATEADD(hour, 6, GETDATE()), 120, 1, 'Scheduled', 2),
79  ('SF305', 'JHB', 'PLZ', DATEADD(hour, 6, GETDATE()), DATEADD(hour, 8, GETDATE()), 100, 0, 'Scheduled', 3);
80  GO
81
82  -- *****
83  --      - Passengers table -
84  -- *****
85
86  -- UserId 4 = pas1, UserId 5 = pas2
87  INSERT INTO Passengers (UserId, PassportNumber, DateOfBirth, Nationality, ContactNumber)
88  VALUES
89  (4, 'PASSPORT001', '1990-05-15', 'South African', '0821234567'),
90  (5, 'PASSPORT002', '1985-08-22', 'South African', '0837654321');
91  GO
92
93  -- *****
94  --      - Bookings table -
95  -- *****
96
97  -- FlightId 1 = SF102, PassengerId 1 = Solomon, PassengerId 2 = John
98
99  INSERT INTO Bookings (FlightId, PassengerId, SeatNumber, BookingStatus, BookingDate)
100  VALUES
101  (1, 1, '12A', 'Confirmed', DATEADD(day, -2, GETDATE())),
102  (1, 2, '14B', 'Confirmed', DATEADD(day, -1, GETDATE()));
103  GO
104
105  -- *****
106  --      - Crew table -
107  -- *****
108
109  -- UserId 2 = agent1 (acting as crew member.)
110
111  INSERT INTO Crew (UserId, CrewType, LicenseNumber, YearsOfExperience, Status)
112  VALUES
113  (2, 'Pilot', 'PIL001', 10, 'Active'),
114  (3, 'CoPilot', 'PIL002', 5, 'Active');
115  GO
116
117
118
119  -- *****
120  -- NB: Run to confirm the data is loaded properly!
121  -- *****
122
123  SELECT * FROM Users;
124  SELECT * FROM Flights;
125  SELECT * FROM Passengers;
126  SELECT * FROM Bookings;

```